

Guia Prático de Arquitetura RAG com Python

Retrieval-Augmented Generation (RAG) é uma técnica que conecta LLMs a fontes de dados externas para gerar respostas contextualizadas, atualizadas e livres de alucinações de forma econômica.

Os 3 Pilares da Arquitetura RAG

- 1. Pipeline de Ingestão:** Processa dados brutos (PDFs, markdown, etc.), divide em blocos menores (chunks) e gera representações numéricas (embeddings) através de uma API.
- 2. Busca Semântica e Reranking:** Converte a pergunta do usuário em vetor e realiza um cálculo de similaridade (como a similaridade de cosseno) para extrair os blocos de texto contextualmente mais relevantes do banco.
- 3. Janela de Contexto Dinâmica:** Consolida o contexto recuperado diretamente dentro do prompt que é enviado para a LLM, garantindo que o modelo use apenas as informações fornecidas para responder.

Implementação Completa em Python

```
import os
import numpy as np
from openai import OpenAI

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY", "sua-chave-aqui"))

DOCUMENTACAO_CHUNKS = [
    "Para instalar o SDK de Pagamentos, execute o comando: npm install @pay-sdk/core.",
    "O erro 403 (Forbidden) ocorre quando as chaves de API estão inválidas ou expiradas.",
    "O erro 500 indica instabilidade interna. Recomenda-se política de retry com backoff exponencial.",
    "Para autenticar as requisições, configure o cliente usando: PayClient(api_key='sua_chave')."
]

def obter_embedding(texto: str, modelo: str = "text-embedding-3-small") -> list:
    resposta = client.embeddings.create(input=[texto], model=modelo)
    return resposta.data.embedding

print("■■ Gerando base de conhecimento vetorial...")
banco_vetorial = {chunk: np.array(obter_embedding(chunk)) for chunk in DOCUMENTACAO_CHUNKS}

def calcular_similaridade_cosseno(vetor_a, vetor_b) -> float:
    return np.dot(vetor_a, vetor_b) / (np.linalg.norm(vetor_a) * np.linalg.norm(vetor_b))

def buscar_contexto_relevante(pergunta_usuario: str, limite_resultados: int = 2) -> list:
    vetor_pergunta = np.array(obter_embedding(pergunta_usuario))
    scores = [(chunk, calcular_similaridade_cosseno(vetor_pergunta, vetor_chunk))
              for chunk, vetor_chunk in banco_vetorial.items()]
    scores.sort(key=lambda x: x[1], reverse=True)
    return [chunk for chunk, score in scores[:limite_resultados]]

pergunta = "Estou recebendo erro 403 ao tentar rodar a API de pagamento. O que pode ser?"
contextos_encontrados = buscar_contexto_relevante(pergunta, limite_resultados=2)
contexto_consolidado = "\n".join([f"- {c}" for c in contextos_encontrados])

resposta_llm = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Responda à pergunta baseando-se estritamente no contexto fornecido."},
```

```
        {"role": "user", "content": f"CONTEXT0:\n{contexto_consolidado}\n\nPERGUNTA:\n{pergunta}"}
    ],
    temperature=0.2
)
print("\n=== RESPOSTA FINAL ===")
print(resposta_llm.choices.message.content)
```

Como obter e configurar sua API Key

1. Acesse o painel de desenvolvedores da [OpenAI API](<https://platform.openai.com/>).
2. Navegue até a aba **API Keys** no menu esquerdo e clique em **+ Create new secret key**.
3. Copie o token imediatamente (ele não será exibido novamente) e guarde em local seguro.
4. No terminal (Linux/macOS), defina a variável com o comando: `export OPENAI_API_KEY='seu_token_aqui'`. No Windows CMD, utilize: `set OPENAI_API_KEY=seu_token_aqui`.